

Tabla de Contenidos

RESUMEN	2
INTRODUCCIÓN	2
DIFICULTADES ESENCIALES DEL SOFTWARE	2
Complejidad	2
Conformidad	2
Cambiabilidad	3
Invisibilidad	3
AVANCES PASADOS QUE RESOLVIERON DIFICULTADES ACCIDENTALES	3
Lenguajes de alto nivel	3
Time-sharing	3
Entornos de programación unificados	3
PROPUESTAS DE “BALAS DE PLATA”	3
Ada y otros lenguajes de alto nivel avanzados	3
Programación Orientada a Objetos (OOP).....	3
Inteligencia Artificial (IA).....	4
Sistemas Expertos.....	4
Programación automática	4
Programación gráfica / visual.....	4
Verificación formal de programas	4
Entornos y herramientas.....	4
Workstations más potentes.....	4
ATAQUES PROMETEDORES A LA ESENCIA CONCEPTUAL	5
Comprar vs Construir - Buy vs Build	5
Refinamiento de requisitos y prototipado rápido.....	5
Desarrollo Incremental – “Cultivar” software en lugar de “Construirlo”	5
Grandes diseñadores - Great Designers.....	5

NO SILVER BULLET

RESUMEN

Todo desarrollo de software incluye 2 tipos de tareas:

- Esenciales: diseñar las estructuras conceptuales complejas que forman el software
- Accidentales: representar esas estructuras en lenguajes de programación y traducirlas a código máquina bajo restricciones de tiempo y espacio.

Sin embargo, eliminar estas tareas accidentales no traerá una mejora x10 en la productividad, al menos que las mismas sean más del 90% del esfuerzo. Por esta razón, el foco debe estar en atacar lo esencial, es decir, cómo diseñar estructuras conceptuales complejas.

Para esto se proponen 4 líneas de acción:

1. Comprar en lugar de construir: aprovechar el mercado de software y no reinventar la rueda
2. Prototipado rápido: iterar en los requisitos usando versiones tempranas del sistema
3. Desarrollo incremental: hacer crecer el software de a poco en lugar de intentar diseñarlo todo de entrada
4. Formar grandes diseñadores: cultivar a quienes tienen capacidad excepcional de diseño

INTRODUCCIÓN

Los monstruos más temidos del folklore son los hombres lobo, porque lo familiar puede transformarse de golpe en horror. Es por esta razón que buscamos la bala de plata que los elimine mágicamente.

De esta forma, los proyectos de software suelen parecer inocentes y simples al principio, pero pueden convertirse en monstruos ya sea por:

- Cronogramas incumplidos
- Presupuestos desbordados
- Productos con fallas

Muchas veces se espera la “bala de plata” en software, ya sea una tecnología o técnica de gestión que reduzca drásticamente los costos.

Pero realmente no existe una bala de plata. Ninguna innovación, por sí sola, va a traer una mejora de 10 veces en productividad, confiabilidad o simplicidad, por lo menos no en la próxima década.

A pesar de esto, sí existen avances e innovaciones alentadoras, ya que el progreso se puede lograr con disciplina, esfuerzo constante y aplicación metódica.

DIFICULTADES ESENCIALES DEL SOFTWARE

Como distinguimos previamente, las dificultades también pueden ser:

- Esenciales: dificultades inevitables, propias de la naturaleza del software
- Accidentes: dificultades externas, que se pueden reducir

Hay cuatro características que hacen que el software sea intrínsecamente difícil:

1. Complejidad

El software es más complejo que cualquier otro producto humano de tamaño comparable, cada parte del software tiende a ser única. Al escalar un sistema, no solo crecen los elementos, también crece de manera no lineal la interacción entre ellos.

La complejidad puede traer consecuencias en distintos aspectos:

- Dificultad de comunicación en los equipos, lo que lleva a errores, sobrecostos y retrasos
- Imposibilidad de prever todos los estados, lo que lleva a baja confiabilidad
- Dificultad para extender funciones sin efectos colaterales
- Riesgos en la seguridad
- Alta dependencia de personas clave

2. Conformidad

El software debe integrarse y adaptarse a otros sistemas e interfaces. Siempre aparece último en la cadena y debe ajustarse a lo ya existente. Esto significa que no hay principios unificadores que simplifiquen el problema.

3. *Cambiabilidad*

El software está en constante presión al cambio debido a que los usuarios encontrar nuevos usos y piden más funciones. A su vez el software sobrevive más que el hardware, por lo que debe adaptarse a nuevas máquinas, discos, pantallas, etc.

Todo software exitoso será modificado, lo que aumentará la complejidad de este a lo largo de su vida útil.

4. *Invisibilidad*

El software no tiene una representación clara. Cuando se intenta diagramar surgen múltiples grafos superpuestos difíciles de visualizar. A pesar de que existan diagramas y abstracciones, la estructura sigue siendo invisible e invisualizable, lo que dificulta el diseño, entorpece la comunicación y limita nuestra capacidad de comprender el sistema en su conjunto.

AVANCES PASADOS QUE RESOLVIERON DIFICULTADES ACCIDENTALES

Hubo tres grandes hitos en la historia del software que mejoraron muchísimo la productividad. Sin embargo, todos ellos atacaron dificultades accidentales, dejando de lado las esenciales.

1. *Lenguajes de alto nivel*

En los primeros tiempos, se programaba en lenguaje máquina, esto hacía que escribir software fuera lento, propenso a errores y difícil de mantener. La aparición de lenguajes de alto nivel implicó un gran salto en la productividad, la confiabilidad y la simplicidad. Esto permitió dejar de pensar en registros y bits, para enfocarse en estructuras de datos, operaciones y lógica, eliminando gran parte de la complejidad accidental.

Sin embargo, este avance ya “toco techo” ya que nuevos lenguajes ofrecen mejoras incrementales, pero no una revolución como la primera transición del ensamblador a lenguajes de alto nivel.

2. *Time-sharing*

Antes, con la programación por lotes, el programador tenía que esperar mucho tiempo entre ejecutar y ver los resultados. Esto cortaba la concentración y hacía perder de vista el contexto.

Con el time-sharing los usuarios podían compartir una computadora grande al mismo tiempo, lo que le permitía al programador probar y ver los resultados inmediatamente, interactuar con el sistema y corregir rápido, aumentando así la productividad al momento de desarrollar software.

Sin embargo, el mayor beneficio de esto fue eliminar la espera, fuera de esto no hay muchas mejoras ya que el factor limitante pasa a ser el tiempo de pensamiento humano, no el de la máquina.

3. *Entornos de programación unificados*

Antes, las herramientas para desarrollar software estaban dispersas, por un lado los compiladores, por otro las librerías, etc. Los entornos como Unix o Interlisp integraron todo en un mismo sistema. Esto permitió mejorar la productividad y potenciar la colaboración entre equipos, ya que los programadores no tenían que luchar con la incompatibilidad de herramientas.

Sin embargo, esto tiene un límite, ya que se siguen resolviendo problemas de “usar las herramientas”, no de diseñar el sistema en sí, es decir, nos ayudan a organizar y automatizar, pero no eliminan la complejidad conceptual del software.

PROPUESTAS DE “BALAS DE PLATA”

Consideremos los desarrollos técnicos que podrían ser tomadas como balas de plata. ¿Qué problemas abarcan? ¿Son problemas provenientes de dificultades esenciales o accidentales?

- *Ada y otros lenguajes de alto nivel avanzados*

El lenguaje de los 80s “Ada” incorporaba modularización, abstracción de datos y jerarquías, prometiendo que los proyectos serían más ordenados y confiables. Su filosofía era positiva, un diseño modular con estructuras jerárquicas y abstracciones, pero en el fondo, es otro lenguaje de alto nivel más.

Es por esto por lo que Brooks dice que Ada es útil, pero no mágica. El gran salto ya había sido pasar del bajo nivel a lenguajes de alto nivel, Ada solo trae mejoras incrementales, no una revolución.

- *Programación Orientada a Objetos (OOP)*

La OOP introduce dos ideas clave:

1. Tipos de datos abstractos: ocultar cómo están implementados los datos y solo exponer operaciones
2. Jerarquías: organizar entidades en árboles (clases y herencia)

Brooks reconoce que la OOP ayuda mucho ya que hace que el diseño sea más natural y cercano al mundo real, reduciendo la “basura sintáctica” que teníamos que escribir en otros lenguajes, pero la complejidad del sistema sigue estando ahí. La OOP no elimina la dificultad de pensar cómo estructurar un sistema complejo.

- *Inteligencia Artificial (IA)*

Se creía que la IA iba a escribir programas automáticamente o iba a entender qué queremos de un sistema, pero en realidad hay dos tipos de IA:

1. IA-1: resuelve con computadoras lo que antes se resolvía con inteligencia humana
2. IA-2: técnicas concretas como sistemas de reglas o heurísticas

Brooks plantea que la IA nos ayuda mucho, pero no cambia el problema real. El problema no está en “como decirle algo a la computadora”, para eso ya tenemos los lenguajes, el problema está en decidir qué queremos que haga el sistema, y a esto, la IA no lo resuelve.

- *Sistemas Expertos*

Son una rama de la IA, los cuales funcionan con una base de reglas y un motor de inferencia que las aplica para dar recomendaciones. Al desarrollar software podrían sugerir cómo testear, aconsejar sobre optimización o detectar errores comunes.

Brooks fundamenta de que pueden ser muy útiles para que el programador novato use el conocimiento de los expertos, ya que ayudan a reducir la brecha entre los mejores programadores y el resto, pero, conseguir que los expertos expliquen bien sus reglas es complicado, por lo que se requiere un gran esfuerzo para armar la base de reglas.

- *Programación automática*

Es la idea soñada: Darle al sistema la especificación del problema y que genere solo el programa.

Brooks dice que esto solo funciona en dominios muy específicos y en los cuales el problema está muy bien definido. Esto se debe a que el software común es demasiado variado, con requisitos ambiguos, casos especiales, interacciones humanas, etc. En el desarrollo de software general, esto no escala, es decir, no hay una “bala de plata” que convierta especificaciones generales en programas completos sin intervención humana.

- *Programación gráfica / visual*

Muchos pensaban que los diagramas y gráficos iban a reemplazar al código, pero Brooks niega esto al fundamentar que:

- Los diagramas de flujo son un mal modelo de los sistemas, ya que terminan siendo enormes, difíciles de comprender y llenos de flechas
- Las pantallas poseen limitaciones físicas
- El software no tiene una geometría clara, ya que cuando quieres dibujar todo, terminas con varios diagramas superpuestos que no dan una visión global clara.

- *Verificación formal de programas*

Se tenía como idea principal demostrar matemáticamente que un programa cumple con su especificación, pero pronto se descubrió que no es una “bala de plata” debido a:

- Altísimo costo
- No se garantiza la corrección total, solo se asegura que el programa cumpla con la especificación dada
- No cumple con el verdadero problema, es cual es la especificación

Lo más difícil no es escribir el código correcto, sino decidir exactamente qué tiene que hacer el software, y esa ambigüedad no la soluciona esta propuesta.

- *Entornos y herramientas*

En los 80s aparecieron entornos integrados que ayudaron al programador con las tareas básicas. Entre ellos tenemos a IDEs, compiladores, debuggers, entre otros.

Brooks reconoce el gran valor que tuvieron ya que facilitaban la vida de los programadores, reducían errores de sintaxis y ayudaban a las organizaciones a aumentar su productividad y confiabilidad. Sin embargo, los beneficios más grandes ya se habían logrado con entornos unificados como Unix. Por lo tanto, esta propuesta solo aportó mejoras menores e incrementales.

- *Workstations más potentes*

En los 80s estaba creciendo la capacidad de las computadoras personales y estaciones de trabajo. Se buscaba aumentar la memoria, agrandar el CPU y compilar más rápido.

Brooks creía que tener más poder de cómputo iba a ser un gran beneficio, pero que no cambiaba el problema esencial. El cuello de botella no era la velocidad de la máquina, sino la cabeza del programador y su capacidad para diseñar, decidir y pensar.

ATAQUES PROMETEDORES A LA ESENCIA CONCEPTUAL

A pesar de que no hay “balas de plata” capaces de generar mejoras mágicas, sí existe la promesa de un progreso constante, aunque no espectacular.

Los “ataques” tecnológicos a los accidentes del proceso de software están fundamentalmente limitados por la ecuación de productividad:

$$\text{tiempo de tarea} = \sum (\text{frecuencia})_i \times (\text{tiempo})_i$$

Si los componentes conceptuales de la tarea me toman la mayor parte del tiempo, ninguna cantidad de actividad sobre los componentes de la tarea puede dar grandes ganancias de productividad. Por lo tanto, debemos considerar aquellos ataques que abordan la esencia del problema del software

Los grandes avances no vendrán de hacer que los programadores escriban código más rápido, sino de encontrar mejores maneras para que piensen y diseñen los programas.

- *Comprar vs Construir - Buy vs Build*

La mejor manera de aumentar la productividad del software es no construirlo desde cero, sino comprarlo cuando ya existe en el mercado. Frente a la expansión del mercado de software se volvió más barato y rápido comprar programas ya hechos que pagar el desarrollo interno. Cada vez existen más productos disponibles para una gran variedad de aplicaciones, y en muchos casos, conviene comprarlos antes que dedicar recursos a desarrollarlos. Compartir los costos de desarrollo entre muchos usuarios, no solo reduce drásticamente el gasto de cada usuario, sino que también multiplica la productividad y cambia radicalmente la economía del software.

- *Refinamiento de requisitos y prototipado rápido*

El aspecto más difícil del desarrollo de software no es programar, sino definir correctamente qué debe hacer el sistema. Los clientes suelen no saber con claridad lo que necesitan, o bien no logran expresarse correctamente hasta ver algo funcionando.

Incluso cuando los clientes creen saber lo que quieren, sus especificaciones son incompletas y contradictorias, lo que puede llevar a un resultado insatisfactorio. Como propuesta, se plantea usar prototipos rápidos que simulen las funciones principales del sistema con el objetivo de que funcione como herramienta de comunicación entre los usuarios y los desarrolladores.

Esto nos permitirá probar y validar las ideas concretas de los clientes, para poder visualizar los problemas de usabilidad y coherencia que no se pueden detectar en un documento de requisitos.

- *Desarrollo Incremental - “Cultivar” software en lugar de “Construirlo”*

Se propone abandonar la metáfora de “construir” software como si fuera un edificio terminado y adoptar la idea de que el software debe crecer como un organismo vivo.

Mediante el enfoque incremental, se busca empezar con una versión mínima que funcione e ir completándola poco a poco. De este modo, siempre existe un sistema operativo (aunque sea limitado).

Esto nos permitirá generar prototipos tempranos, y en caso de que sea necesario, retroceder y corregir fácilmente. A su vez, nos ayuda a elevar la moral del equipo al ver resultados tangibles (aunque simples), acelerando el trabajo y aumentando la motivación de cada integrante del equipo.

- *Grandes diseñadores - Great Designers*

El avance del software no depende solo de métodos y herramientas, sino del talento de los diseñadores.

El foco central está en las personas involucradas en el desarrollo de software. Los mejores diseñadores crean sistemas más simples, elegantes y eficientes, y lo hacen con menos esfuerzo. Es por esto que la empresa debería invertir en identificar y formar grandes diseñadores con la misma seriedad con la que entrenan a sus gerentes.

Es muy importante detectar el talento tempranamente, incluso si no tiene mucha experiencia. Para ello también se debe asignar mentores y planes de carrera que ayudarán a la formación de los novatos.